Model Description

Data loading:

Looped through all XML files and stored "filename", "building_name", image coordinates ("img_width", "img_height") and bounding box coordinates ("xmin", "ymin", "xmax", "ymax") in a dataframe.

```
df.head()
```

	filename	building_name	img_width	img_height	xmin	ymin	xmax	ymax
0	A_1.JPG	Block A	4032	3024	14	1466	4031	2701
1	A_10.JPG	Block A	4032	3024	346	230	4032	2665
2	A_11.jpg	Block A	4032	3024	278	637	4031	2897
3	A_12.jpg	Block A	4032	3024	14	653	3856	3014
4	A_13.jpg	Block A	4032	3024	433	495	4031	2966

Number of rows in dataframe:

```
len(df)
```

```
101
```

Data preprocessing:

Each image entry in the dataframe is processed in multiple ways to enhance model generalization. For every original image, we generated eight samples for **training set** (original, cropped, 3 augmentations of original, 3 augmentations of cropped). We applied following transformations for data augmentation of training set:

- **RandomResizedCrop:** Randomly crop a region from the image (70% to 100% of the original image size) and resize it to 224x224 (standard input size for MobileNet)
- **RandomHorizontalFlip:** Randomly flip the image horizontally
- **RandomRotation:** Randomly apply ±15° rotation on images
- **ColorJitter:** Randomly change brightness, contrast, saturation, and hue of the image
- **RandomAdjustSharpness:** Adjust the image sharpness

Then we standardized each image by subtracting the mean and dividing by the standard deviation for each color channel.

```
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    transforms.RandomAdjustSharpness(sharpness_factor=2, p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

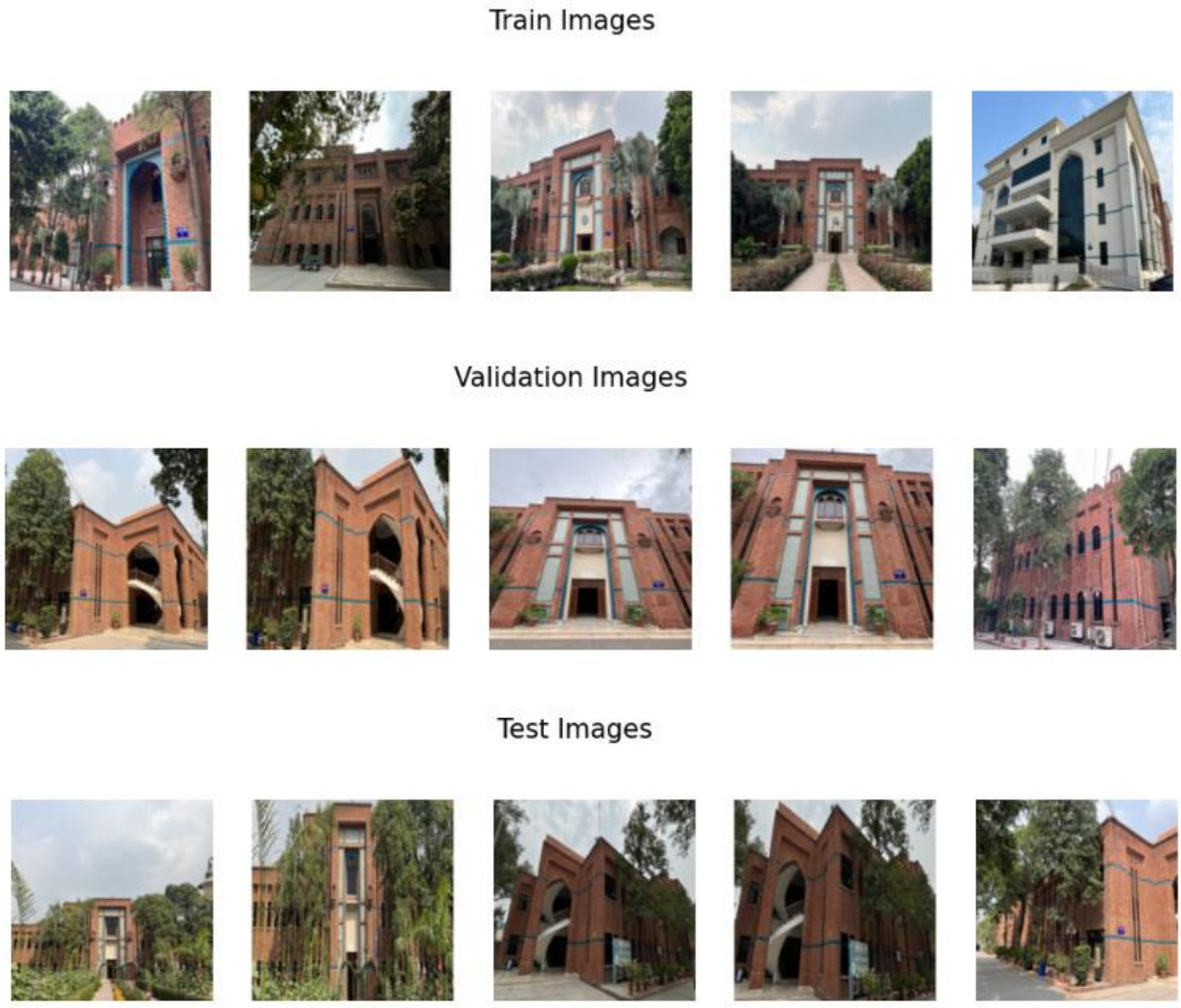
For **validation and test set**, we resized the images to 224x224 (standard input size for MobileNet), applied normalization and generated two samples for each image (original, cropped). This ensures that the model is evaluated on clean, unaltered data.

```
val_test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

We divided the data in training and validation sets class-wise. Then from each class we took 73% training samples, 15% for validation and remaining for testing to avoid class imbalance.

```
Train Samples: 568, Validation Samples: 24, Test Samples: 36
```

Visualization:



Model Training:

We used MobileNet V2 for finetuning as it is designed to be highly efficient, with fewer parameters and lower computational cost compared to heavier models like ResNet or VGG. It helps in avoiding overfitting and allows the model to generalize better on small dataset. Since there are 6 building types, so we replaced the final fully connected layer in the model with a new linear layer that outputs 6 class scores. We used Cross Entropy Loss (suitable for multi-class classification) and Adam optimizer with learning rate=0.0001 for stable training. The model is trained for 10 epochs.

```
Epoch 1/10, Train Loss: 1.5921, Train Acc: 50.00%, Val Loss: 1.3630, Val Acc: 70.83%
Model saved
Epoch 2/10, Train Loss: 1.0322, Train Acc: 84.86%, Val Loss: 0.7889, Val Acc: 87.50%
Model saved
Epoch 3/10, Train Loss: 0.5468, Train Acc: 94.54%, Val Loss: 0.4226, Val Acc: 91.67%
Model saved
Epoch 4/10, Train Loss: 0.2699, Train Acc: 96.13%, Val Loss: 0.2413, Val Acc: 100.00%
Model saved
Epoch 5/10, Train Loss: 0.1421, Train Acc: 99.12%, Val Loss: 0.1279, Val Acc: 100.00%
Model saved
Epoch 6/10, Train Loss: 0.0725, Train Acc: 99.82%, Val Loss: 0.0971, Val Acc: 100.00%
Model saved
Epoch 7/10, Train Loss: 0.0620, Train Acc: 99.30%, Val Loss: 0.0688, Val Acc: 100.00%
Model saved
Epoch 8/10, Train Loss: 0.0541, Train Acc: 99.12%, Val Loss: 0.0515, Val Acc: 100.00%
Model saved
Epoch 9/10, Train Loss: 0.0267, Train Acc: 99.82%, Val Loss: 0.0487, Val Acc: 100.00%
Model saved
Epoch 10/10, Train Loss: 0.0207, Train Acc: 100.00%, Val Loss: 0.0427, Val Acc: 100.00%
Model saved
```

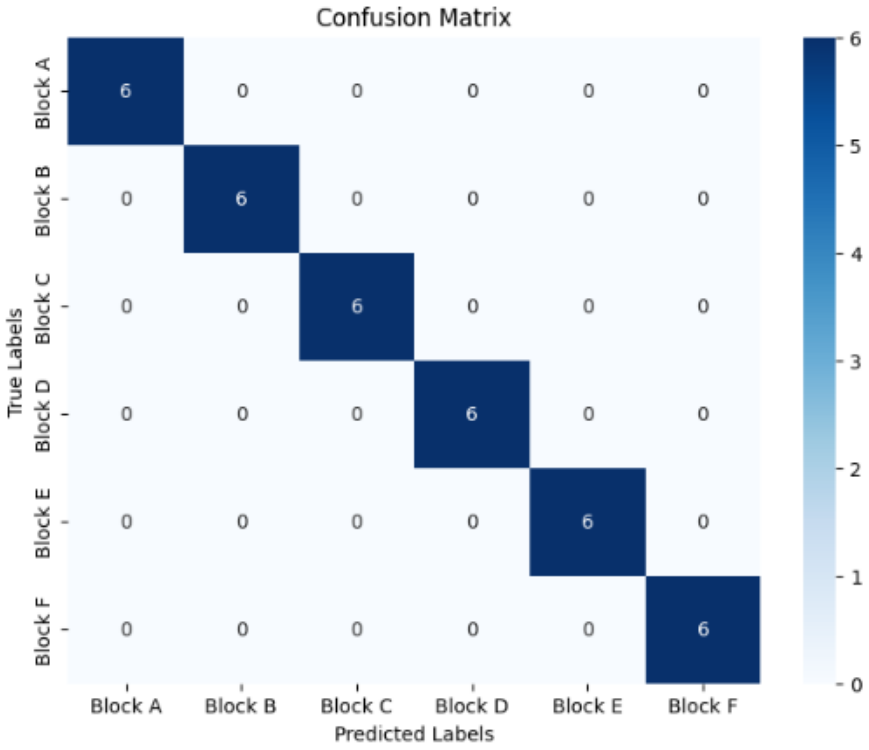
Evaluation:

```
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
```

These metrics tell that the model was able to predict all classes correctly i.e 100% accuracy.

Classification Report

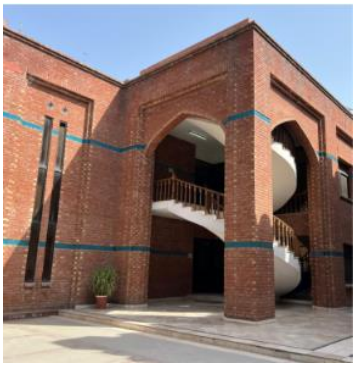
	precision	recall	f1-score	support
Block A	1.00	1.00	1.00	6
Block B	1.00	1.00	1.00	6
Block C	1.00	1.00	1.00	6
Block D	1.00	1.00	1.00	6
Block E	1.00	1.00	1.00	6
Block F	1.00	1.00	1.00	6
accuracy			1.00	36
macro avg	1.00	1.00	1.00	36
weighted avg	1.00	1.00	1.00	36



Inference:

Testing model on new, unseen data.

Image 1:



```
predict_building(img_1_path)
```

```
Actual Building: Block A
Predicted Building: Block A
```

Image 2:

```
predict_building(img_2_path)
```

```
Actual Building: Block E
Predicted Building: Block E
```

Image 3:

```
predict_building(img_3_path)
```

```
Actual Building: Block C
Predicted Building: Block C
```

Image 4:

```
predict_building(img_4_path)
```

```
Actual Building: Block D
Predicted Building: Block D
```

The model was able to predict all the classes correctly.